

Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4

По дисциплине: «Аппаратно-программное обеспечение ЭВМ и сетей»

Тема: «Архитектура и программирование сопроцессора. Использование
целочисленных и вещественных команд»

Выполнила:

Студентка 4 курса

Группы АС-66

Прокурат В.Д.

Проверила:

Кулик А.Д.

Цель работы: изучение архитектуры и программной модели сопроцессора; практическая работа с системой целочисленных и вещественных команд.

Вариант 20 (2)

Постановка задачи. Раздел 1.

1. Изучить следующие теоретические сведения: архитектура и программная модель сопроцессора – регистры и их назначение; форматы чисел, типовые команды, целочисленные команды (используя материалы лекций и методические указания).
2. Составить и отладить программу на языке ассемблера для вычисления значения функции, используя регистры сопроцессора и целочисленные команды согласно варианту. Оформить ее в виде отдельной функции. Каждую ветку алгоритма оформить в виде отдельной внутренней процедуры. Ввод и вывод данных осуществляется через консоль. Программу протестировать по всем условиям.

Раздел 2. 1. Изучить систему вещественных команд сопроцессора (используя материалы лекций и методические указания).

2. Составить и отладить программу на языке ассемблера для вычисления значения функции, используя регистры сопроцессора и вещественные команды согласно варианту. Оформить ее в виде отдельной функции. Каждую ветку алгоритма оформить в виде отдельной внутренней процедуры. Ввод и вывод данных осуществляется через консоль.

Задание:

$$\text{Вариант 2}$$
$$y = \begin{cases} 4xy + 5, & x + y > 10; \\ \frac{3xy + 2y}{x^2 + y^2 + 1}, & x + y < -2; \\ 6x - 2y^2 + 1, & -2 \leq x + y \leq 1. \end{cases}$$

Код программы:

```
.686                                ; нужен для fcomip (процессоры P6+)
.model flat, c
option casemap:none

extern printf:near
extern scanf:near

.data
    prompt_i  db 10, "=== Part 1: integer FPU instructions ===", 10
               db "Enter integers x y: ", 0
    prompt_r  db 10, "=== Part 2: real FPU instructions ===", 10
               db "Enter real x y (use dot): ", 0
    fmt_int   db "%d %d", 0
    fmt_dbl   db "%lf %lf", 0
    fmt_out   db "Branch %d: y = %.6f", 10, 0
    msg_undef db "Function is undefined (1 < x+y <= 10)", 10, 0

; целые константы (для fimul / fiadd, формат DWORD)
i1  dd 1
i2  dd 2
i3  dd 3
i4  dd 4
i5  dd 5
```

```

i6    dd 6
i10   dd 10
im2   dd -2

```

```

; вещественные константы (REAL8 = double)

```

```

r1    real8 1.0
r3    real8 3.0
r4    real8 4.0
r5    real8 5.0
r6    real8 6.0
r10   real8 10.0
rm2   real8 -2.0

```

```

.data?

```

```

xi    dd ?                ; целые x, y и их сумма
yi    dd ?
sumi  dd ?
xr    real8 ?             ; вещественные x, y и их сумма
yr    real8 ?
sumr  real8 ?

```

```

.code

```

```

; -----
; РАЗДЕЛ 1. func_int: целочисленные команды
; Вход: xi, yi (память). Выход: eax = номер ветки (0 - не определена),
; результат в ST(0) (если eax != 0).
; -----

```

```

func_int proc
    fild  dword ptr [xi]      ; ST0 = x
    fiadd dword ptr [yi]      ; ST0 = x + y
    fistp dword ptr [sumi]    ; сумма как целое, стек пуст
    mov   eax, [sumi]
    cmp   eax, 10
    jg    fi_b1               ; x+y > 10
    cmp   eax, -2
    jl    fi_b2               ; x+y < -2
    cmp   eax, 1
    jle   fi_b3               ; -2 <= x+y <= 1
    xor   eax, eax            ; иначе - не определена
    ret
fi_b1:
    call  int_br1
    mov   eax, 1
    ret
fi_b2:
    call  int_br2
    mov   eax, 2
    ret
fi_b3:
    call  int_br3
    mov   eax, 3
    ret
func_int endp

```

```

; Ветка 1: 4xy + 5

```

```

int_br1 proc
    fild  dword ptr [xi]      ; x
    fimul dword ptr [yi]      ; x*y
    fimul dword ptr [i4]      ; 4xy
    fiadd dword ptr [i5]      ; 4xy + 5
    ret
int_br1 endp

```

```

; Ветка 2: (3xy + 2y) / (x^2 + y^2 + 1)
; Сначала знаменатель (окажется в ST1), затем числитель (ST0).
int_br2 proc

```

```

    fild dword ptr [xi]
    fimul dword ptr [xi]          ; x^2
    fild dword ptr [yi]
    fimul dword ptr [yi]          ; y^2
    faddp                                     ; x^2 + y^2
    fiadd dword ptr [i1]          ; ST0 = знаменатель
    fild dword ptr [xi]
    fimul dword ptr [yi]
    fimul dword ptr [i3]          ; 3xy
    fild dword ptr [yi]
    fimul dword ptr [i2]          ; 2y
    faddp                                     ; ST0 = числитель, ST1 = знаменатель
    fdiv st(0), st(1)             ; ST0 = числитель / знаменатель
    fstp st(1)                    ; убрать знаменатель, результат в ST0
    ret
int_br2 endp

```

; Ветка 3: $6x - 2y^2 + 1$

```

int_br3 proc
    fild dword ptr [yi]
    fimul dword ptr [yi]          ; y^2
    fimul dword ptr [i2]          ; 2y^2
    fchs                             ; -2y^2
    fild dword ptr [xi]
    fimul dword ptr [i6]          ; 6x
    faddp                                     ; 6x - 2y^2
    fiadd dword ptr [i1]          ; + 1
    ret
int_br3 endp

```

```

; -----
; РАЗДЕЛ 2. func_real: вещественные команды
; Вход: xr, yr (память). Выход: eax = номер ветки, результат в ST(0).
; -----

```

```

func_real proc
    fld qword ptr [xr]
    fadd qword ptr [yr]
    fstp qword ptr [sumr]          ; sum = x + y

    fld qword ptr [r10]           ; ST1 = 10
    fld qword ptr [sumr]          ; ST0 = sum
    fcomip st(0), st(1)           ; сравнить sum с 10, снять sum
    fstp st(0)                    ; снять 10 (флаги не меняются)
    ja fr_b1                      ; sum > 10

    fld qword ptr [rm2]
    fld qword ptr [sumr]
    fcomip st(0), st(1)
    fstp st(0)
    jb fr_b2                      ; sum < -2

    fld qword ptr [r1]
    fld qword ptr [sumr]
    fcomip st(0), st(1)
    fstp st(0)
    jbe fr_b3                     ; sum <= 1 (и уже >= -2)

    xor eax, eax                  ; не определена
    ret

fr_b1:
    call real_br1
    mov eax, 1
    ret

fr_b2:
    call real_br2
    mov eax, 2
    ret

```

```

fr_b3:
    call real_br3
    mov  eax, 3
    ret
func_real endp

; Ветка 1: 4xy + 5
real_br1 proc
    fld  qword ptr [xr]
    fmul qword ptr [yr]      ; xy
    fmul qword ptr [r4]      ; 4xy
    fadd qword ptr [r5]      ; 4xy + 5
    ret
real_br1 endp

; Ветка 2: (3xy + 2y) / (x^2 + y^2 + 1)
real_br2 proc
    fld  qword ptr [xr]
    fmul st(0), st(0)        ; x^2
    fld  qword ptr [yr]
    fmul st(0), st(0)        ; y^2
    faddp
    fadd qword ptr [r1]      ; ST0 = знаменатель
    fld  qword ptr [xr]
    fmul qword ptr [yr]
    fmul qword ptr [r3]      ; 3xy
    fld  qword ptr [yr]
    fadd st(0), st(0)        ; 2y
    faddp
    fdiv st(0), st(1)        ; числитель / знаменатель
    fstp st(1)               ; результат в ST0
    ret
real_br2 endp

; Ветка 3: 6x - 2y^2 + 1
real_br3 proc
    fld  qword ptr [yr]
    fmul st(0), st(0)        ; y^2
    fadd st(0), st(0)        ; 2y^2
    fchs
    fmul qword ptr [xr]
    fmul qword ptr [r6]      ; 6x
    faddp
    fadd qword ptr [r1]      ; + 1
    ret
real_br3 endp

main proc
    ; ---- Раздел 1 ----
    push offset prompt_i
    call printf
    add  esp, 4

    push offset yi           ; scanf("%d %d", &xi, &yi)
    push offset xi
    push offset fmt_int
    call scanf
    add  esp, 12

    call func_int
    test eax, eax
    jz   undef1
    sub  esp, 8              ; место под double в стеке
    fstp qword ptr [esp]    ; результат из ST0 прямо в стек
    push eax                ; номер ветки
    push offset fmt_out
    call printf

```

```

    add    esp, 16                ; 4 + 4 + 8
    jmp    part2
undef1:
    push   offset msg_undef
    call   printf
    add    esp, 4

    ; ---- Раздел 2 ----
part2:
    push   offset prompt_r
    call   printf
    add    esp, 4

    push   offset yr              ; scanf("%lf %lf", &xr, &yr)
    push   offset xr
    push   offset fmt_dbl
    call   scanf
    add    esp, 12

    call   func_real
    test   eax, eax
    jz     undef2
    sub    esp, 8
    fstp   qword ptr [esp]
    push   eax
    push   offset fmt_out
    call   printf
    add    esp, 16
    jmp    done
undef2:
    push   offset msg_undef
    call   printf
    add    esp, 4
done:
    xor    eax, eax              ; return 0
    ret
main endp

end

```

```

x86 Native Tools Command Pi
D:\politex\ЛАБЫ\7 семестр\АПО ЭВМ и C\lab4>lab4.exe

=== Part 1: integer FPU instructions ===
Enter integers x y: 2 -1
Branch 3: y = 11.000000

=== Part 2: real FPU instructions ===
Enter real x y (use dot): 2.5 -1.5
Branch 3: y = 11.500000

D:\politex\ЛАБЫ\7 семестр\АПО ЭВМ и C\lab4>lab4.exe

=== Part 1: integer FPU instructions ===
Enter integers x y: -1 -1
Branch 3: y = -7.000000

=== Part 2: real FPU instructions ===
Enter real x y (use dot): -1.5 -0.6
Branch 2: y = 0.415512

D:\politex\ЛАБЫ\7 семестр\АПО ЭВМ и C\lab4>lab4.exe

=== Part 1: integer FPU instructions ===
Enter integers x y: 1 0
Branch 3: y = 7.000000

=== Part 2: real FPU instructions ===
Enter real x y (use dot): 0.6 0.5
Function is undefined (1 < x+y <= 10)

D:\politex\ЛАБЫ\7 семестр\АПО ЭВМ и C\lab4>

```

Вывод: в ходе лабораторной работы изучены архитектура и программная модель математического сопроцессора x87: стек из восьми 80-битных регистров ST(0)–ST(7), порядок загрузки и выгрузки данных, вычисления через вершину стека, сравнение чисел командой `fcomip` с передачей результата в флаги EFLAGS и условными переходами. Практически освоены две системы команд. Целочисленные (`fild`, `fiadd`, `fmul`, `fistp`) берут операнды-целые из памяти и сами переводят их в вещественный формат, но не учитывают дробную часть данных. Вещественные (`fld`, `fadd`, `fmul`, `fdiv`, `fchs`) работают с форматом `double`, и от дробной части зависят и значение функции, и выбор ветки.